

Gender dynamics experiments

Eduardo García-Portugués, Carlos Vaquero

2026-01-20

Running experiment 3 with subset Sop_features_non_cat

Load libraries

```
# Load libraries
library(knitr)
library(dplyr)
library(recipes)
library(rsample)
library(yardstick)
library(glmnet)
library(ggplot2)
library(caret)
library(tibble)
sessionInfo()
```

```
R version 4.5.2 (2025-10-31)
Platform: aarch64-apple-darwin20
Running under: macOS Sequoia 15.7
```

```
Matrix products: default
```

```
BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/
```

```
LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK ve
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: Europe/Madrid
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods   base
```

other attached packages:

```
[1] tibble_3.3.1    caret_7.0-1      lattice_0.22-7   ggplot2_4.0.1  
[5] glmnet_4.1-10   Matrix_1.7-4     yardstick_1.3.2  rsample_1.3.1  
[9] recipes_1.3.1   dplyr_1.1.4      knitr_1.51
```

loaded via a namespace (and not attached):

```
[1] gtable_0.3.6      shape_1.4.6.1    xfun_0.55  
[4] vctrs_0.6.5       tools_4.5.2      generics_0.1.4  
[7] stats4_4.5.2      parallel_4.5.2   ModelMetrics_1.2.2.2  
[10] pkgconfig_2.0.3   data.table_1.18.0 RColorBrewer_1.1-3  
[13] S7_0.2.1          lifecycle_1.0.5  stringr_1.6.0  
[16] compiler_4.5.2    farver_2.1.2     codetools_0.2-20  
[19] htmltools_0.5.9   class_7.3-23     yaml_2.3.12  
[22] prodlim_2025.04.28 pillar_1.11.1    furrr_0.3.1  
[25] tidyr_1.3.2       MASS_7.3-65      gower_1.0.2  
[28] iterators_1.0.14  rpart_4.1.24     foreach_1.5.2  
[31] nlme_3.1-168      parallelly_1.46.1 lava_1.8.2  
[34] tidyselect_1.2.1  digest_0.6.39    stringi_1.8.7  
[37] future_1.68.0     reshape2_1.4.5   purrr_1.2.1  
[40] listenv_0.10.0    splines_4.5.2    fastmap_1.2.0  
[43] grid_4.5.2        cli_3.6.5        magrittr_2.0.4  
[46] survival_3.8-3    future.apply_1.20.1 withr_3.0.2  
[49] scales_1.4.0      lubridate_1.9.4  timechange_0.3.0  
[52] rmarkdown_2.30    globals_0.18.0   otel_0.2.0  
[55] nnet_7.3-20       timeDate_4051.111 evaluate_1.0.5  
[58] hardhat_1.4.2     rlang_1.1.7      Rcpp_1.1.1  
[61] glue_1.8.0        pROC_1.19.0.1    ipred_0.9-15  
[64] jsonlite_2.0.0    plyr_1.8.9       R6_2.6.1
```

Read datasets

```
# Load csv  
df <- read.csv(paste0("Experiment_", experiment, ".csv"),  
               stringsAsFactors = TRUE)  
  
# Choose target column based on experiment  
target_col <- case_when(  
  experiment %in% c(1, 2) ~ "Gender",  
  experiment == 3 ~ "Singer_Sex"  
)  
  
# Rename target to 'y'  
df <- df |>  
  rename(y = !!target_col)  
  
# Number of observations per group
```

```
df |>
  group_by(y) |>
  summarise(n = n(), .groups = "drop")
```

```
# A tibble: 2 x 2
```

```
  y         n
  <fct> <int>
1 Female  644
2 Male    792
```

```
# Split into train / test (85% / 15%)
set.seed(123)
split_obj <- initial_split(df, prop = 0.85)
train <- training(split_obj)
test <- testing(split_obj)
```

```
# Determine subset based ONLY on training data
features_list_train <- get_features_subsets_voice(train)
if (!(subset_name %in% names(features_list_train))) {
  stop("Subset ", subset_name, " not found")
}

# Select features based on subset name
selected_features <- colnames(features_list_train[[subset_name]])
train <- train |>
  select(all_of(selected_features), y)
test <- test |>
  select(all_of(selected_features), y)
dim(train)
```

```
[1] 1220  87
```

```
dim(test)
```

```
[1] 216  87
```

```
# Combined dataset for recipes
combined <- bind_rows(train, test)
```

Preprocessing recipes

```

# Preprocessing recipe
preprocess_recipe <- function(data) {
  recipe(y ~ ., data = data) |>
    # Preprocess all predictors BEFORE dummifying
    step_zv(all_predictors()) |>
    step_nzv(all_predictors()) |>
    step_filter_missing(all_predictors(), threshold = 0.05) |>
    step_impute_knn(all_predictors(), neighbors = 5) |>
    # Collapse rare factor levels (BEFORE dummy creation)
    step_other(all_nominal_predictors(), threshold = 0.10) |>
    # Preprocess numeric predictors only
    step_corr(all_numeric_predictors(), threshold = 0.80) |>
    step_lincomb(all_numeric_predictors(), max_steps = 20) |>
    step_normalize(all_numeric_predictors()) |>
    # Create dummy variables (AFTER normalization)
    # Use one-hot encoding such that no dummy variable is dropped and we can
    # later drop the "other" level manually (so it becomes the reference level
    # and we avoid hard-to-explain dummies in the final model)
    # step_dummy(all_nominal_predictors(), one_hot = TRUE) |>
    step_dummy(all_nominal_predictors(), one_hot = FALSE) |>
    # Drop the dummies for the "other" level
    step_rm(ends_with("_other")) |>
    # Post-dummy cleanup
    step_zv(all_predictors()) |>
    step_nzv(all_predictors()) |>
    step_lincomb(all_predictors(), max_steps = 20)
}

```

```

# Train recipe using train data
rec_train <- preprocess_recipe(data = train)
prep_rec_train <- prep(rec_train)

# Bake train data
train_bake <- bake(prep_rec_train, new_data = NULL)
test_train_bake <- bake(prep_rec_train, new_data = test)

# Train recipe using combined data (train + test)
rec_all <- preprocess_recipe(data = combined)
prep_all <- prep(rec_all)

# Fully baked dataset for the final model fit
all_bake <- bake(prep_all, new_data = NULL)

# Export datasets for future reference and use
dir.create("preprocessed_datasets", showWarnings = FALSE)
write.csv(train_bake, paste0("preprocessed_datasets/train_bake_exp", experiment,
                             "_sub_", subset_name, ".csv"),
          row.names = FALSE)

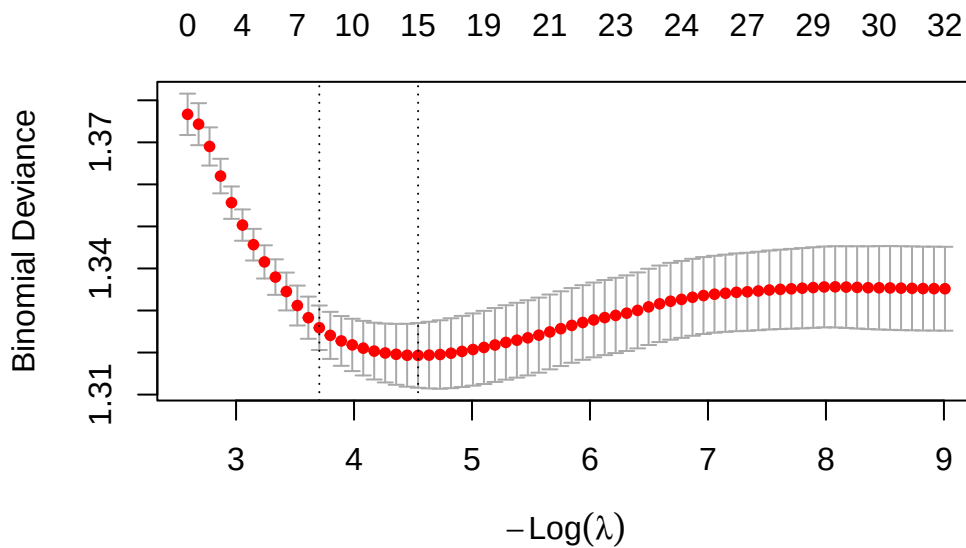
```

```
write.csv(test_train_bake, paste0("preprocessed_datasets/test_train_bake_exp",
                                experiment, "_sub_", subset_name, ".csv"),
          row.names = FALSE)
write.csv(all_bake, paste0("preprocessed_datasets/all_bake_", experiment,
                           "_sub_", subset_name, ".csv"),
          row.names = FALSE)
```

Lasso logistic

Fitting

```
# Fit by 10-fold cross-validation
set.seed(42)
train_bake_no_y <- train_bake |>
  select(-y) |>
  as.matrix()
cv1_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$y,
                      family = "binomial", alpha = 1, nfolds = 10)
plot(cv1_train)
```



Predictions

```
# Predictions
test_train_bake_no_y <- test_train_bake |>
  select(-y) |>
```

```

as.matrix()
y_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "class")
p_test_hat_LL <- predict(cv1_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "response")

```

Metrics

```

# Metrics
acc_LL <- mean(test$y == y_test_hat_LL)
auc_LL <- auc_binary(test$y, p_test_hat_LL)
f1_LL <- f1(test$y, y_test_hat_LL)
data.frame("Accuracy" = acc_LL, "AUC" = auc_LL, "Macro_F1" = f1_LL) |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1
0.5833	0.6359	0.5533

Relevant coefficients

```

# Extract nonzero coefficients for lambda.1se and order them by absolute value
coefs1_train <- predict(cv1_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs1_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  kable(digits = 4)

```

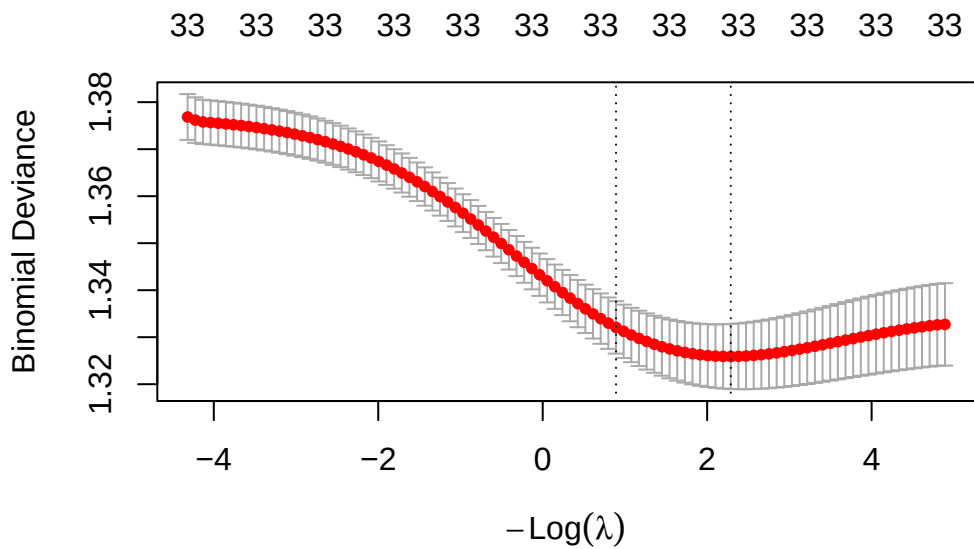
feature	coefficient
PartSop_HighestNoteIndex	-0.2445
(Intercept)	0.2114
PartSop_LargestSemitonesAsc	0.1450
PartSop_DescendingIntervallicStd	0.0914
PartSop_Ambitus	0.0748
PartSop_LargestSemitonesDesc	-0.0608
PartSop_IntervalsMinorAll_Per	-0.0583
PartSop_IntervalsBeyondOctaveAsc_Per	0.0155
PartSop_AbsoluteIntervallicTrimRatio	0.0045

feature	coefficient
---------	-------------

Ridge logistic model

Fitting

```
# Fit by 10-fold cross-validation
set.seed(42)
cv0_train <- cv.glmnet(x = train_bake_no_y, y = train_bake$y,
                      family = "binomial", alpha = 0, nfolds = 10)
plot(cv0_train)
```



Predictions

```
# Predictions
y_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "class")
p_test_hat_RL <- predict(cv0_train, newx = test_train_bake_no_y,
                        s = "lambda.1se", type = "response")
```

Metrics

```
# Metrics
acc_RL <- mean(test$y == y_test_hat_RL)
auc_RL <- auc_binary(test$y, p_test_hat_RL)
f1_RL <- f1(test$y, y_test_hat_RL)
data.frame("Accuracy" = acc_RL, "AUC" = auc_RL, "Macro_F1" = f1_RL) |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1
0.5972	0.6583	0.5543

Relevant coefficients

```
# Extract coefficients and sort them by absolute value
coefs0_train <- predict(cv0_train, s = "lambda.1se", type = "coefficients") |>
  as.matrix() |>
  as.data.frame() |>
  rownames_to_column(var = "feature") |>
  rename("coefficient" = "lambda.1se")
coefs0_train |>
  filter(coefficient != 0) |>
  arrange(desc(abs(coefficient))) |>
  head(30) |>
  kable(digits = 4)
```

feature	coefficient
(Intercept)	0.2329
PartSop_HighestNoteIndex	-0.1109
PartSop_LargestSemitonesAsc	0.0607
PartSop_LargestIntervalAll_P8	-0.0561
PartSop_AbsoluteIntervallicTrimRatio	0.0529
PartSop_LargestSemitonesDesc	-0.0529
PartSop_IntervalsMinorAll_Per	-0.0526
PartSop_DescendingIntervallicStd	0.0511
PartSop_Ambitus	0.0423
PartSop_IntervalsBeyondOctaveAsc_Per	0.0420
PartSop_IntervalsPerfectDesc_Per	0.0411
PartSop_TrimmedAbsoluteIntervallicStd	0.0365
PartSop_IntervallicMean	-0.0339
PartSop_IntervallicKurtosis	0.0334
PartSop_LeapsAsc_Per	0.0284
PartSop_IntervalsDiminishedDesc_Per	-0.0275
PartSop_IntervalsPerfectAsc_Per	0.0262

feature	coefficient
PartSop_IntervalsBeyondOctaveDesc_Per	0.0239
PartSop_IntervalsAugmentedDesc_Per	0.0233
PartSop_DottedRhythm	-0.0192
PartSop_IntervalsMajorAll_Per	0.0181
PartSop_IntervalsPerfectAll_Per	0.0177
PartSop_IntervalsAugmentedAsc_Per	0.0150
PartSop_IntervalsWithinOctaveDesc_Per	0.0144
PartSop_VoicePresence	0.0125
PartSop_RhythmInt	0.0124
PartSop_StepwiseMotionAll_Per	-0.0098
PartSop_IntervallicTrimDiff	0.0078
PartSop_IntervalsMajorAsc_Per	0.0059
PartSop_LeapsDesc_Per	0.0052

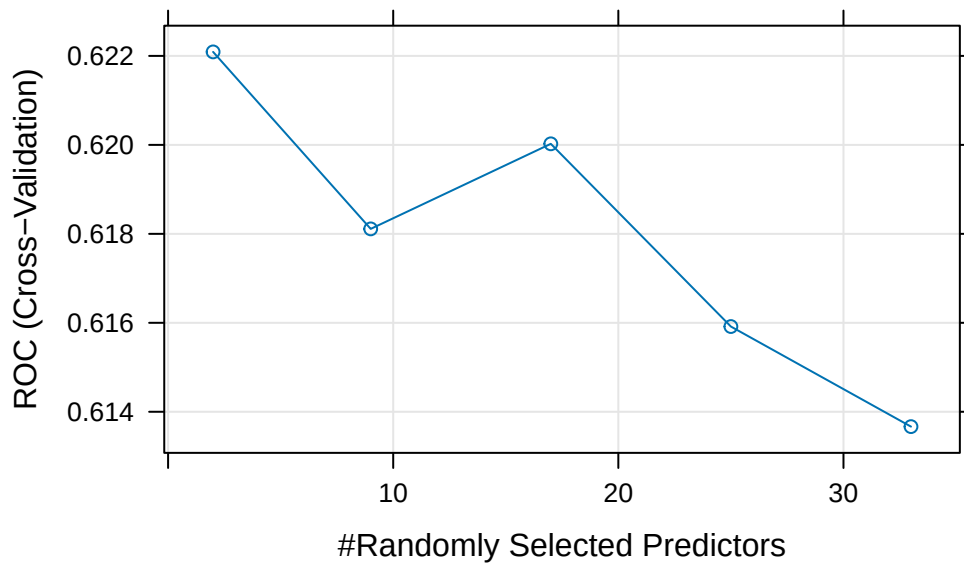
Random forests

Fitting

```
# Set seeds for reproducibility in caret with RF
set.seed(42)
n_folds <- 10
n_tune <- 5
rf_seeds <- c(
  lapply(seq_len(n_folds), function(i) rep(i, n_tune)),
  list(999L)
)

# Define 10-fold CV
cv_ctrl <- trainControl(method = "cv", number = 10, classProbs = TRUE,
  summaryFunction = twoClassSummary,
  savePredictions = "final", seeds = rf_seeds)

# Train Random Forest with tuning of mtry optimizing AUC
rf_train <- train(x = train_bake_no_y, y = train_bake$y, method = "rf",
  metric = "ROC", trControl = cv_ctrl, tuneLength = n_tune)
plot(rf_train)
```



Predictions

```
# Predictions
y_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y)
positive_class <- levels(train_bake$y)[2] # "Class 1" is the second factor level
p_test_hat_RF <- predict(rf_train, newdata = test_train_bake_no_y,
  type = "prob")[, positive_class]
```

Metrics

```
# Metrics
acc_RF <- mean(test_train_bake$y == y_test_hat_RF)
auc_RF <- auc_binary(test$y, p_test_hat_RF)
f1_RF <- f1(test$y, y_test_hat_RF)
data.frame("Accuracy" = acc_RF, "AUC" = auc_RF, "Macro_F1" = f1_RF) |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1
0.6111	0.6592	0.5909

Variables importance

```
# Variables importance
rf_importance <- varImp(rf_train)$importance
rf_importance |>
  rownames_to_column("feature") |>
  arrange(desc(Overall)) |>
  head(30) |>
  kable(digits = 4)
```

feature	Overall
PartSop_DescendingIntervallicStd	100.0000
PartSop_IntervallicKurtosis	95.0468
PartSop_AbsoluteIntervallicTrimRatio	94.9705
PartSop_IntervalsMinorAll_Per	94.7069
PartSop_IntervalsPerfectDesc_Per	93.0771
PartSop_VoicePresence	92.5720
PartSop_LeapsAsc_Per	91.7735
PartSop_TrimmedAbsoluteIntervallicStd	90.2466
PartSop_IntervalsPerfectAsc_Per	89.5791
PartSop_IntervalsWithinOctaveDesc_Per	88.8861
PartSop_SyllabicRatio	88.6930
PartSop_RhythmInt	88.6212
PartSop_IntervalsPerfectAll_Per	87.3035
PartSop_IntervalsMajorAsc_Per	86.8829
PartSop_IntervalsMajorAll_Per	86.2841
PartSop_LeapsDesc_Per	85.6963
PartSop_AverageDuration	84.2849
PartSop_IntervallicTrimDiff	83.4143
PartSop_StepwiseMotionAll_Per	80.6727
PartSop_IntervallicMean	77.6534
PartSop_IntervallicTrimRatio	75.8113
PartSop_HighestNoteIndex	74.4837
PartSop_DottedRhythm	69.1628
PartSop_Ambitus	59.9900
PartSop_IntervalsAugmentedAsc_Per	59.4023
PartSop_IntervalsBeyondOctaveAsc_Per	57.6281
PartSop_IntervalsDiminishedDesc_Per	56.3526
PartSop_IntervalsDiminishedAsc_Per	51.4098
PartSop_LargestSemitonesDesc	44.9208
PartSop_LargestSemitonesAsc	44.8122

Metrics for all models on test data

```

# Save predictions for all models
dir.create("results/y_hats", showWarnings = FALSE)
datasets_to_save <- list(
  RL = y_test_hat_RL,
  LL = y_test_hat_LL,
  RF = y_test_hat_RF,
  y_true = test$y
)
for (name in names(datasets_to_save)) {

  data_to_write <- datasets_to_save[[name]]

  # If it's a vector, convert to data frame
  if (is.vector(data_to_write)) {
    data_to_write <- data.frame(prediction = data_to_write)
  }

  # Construct filename
  file_name <- paste0("results/y_hats/", name, "_", experiment,
    "_sub_", subset_name, ".csv")

  # Write CSV
  write.csv(data_to_write, file_name, row.names = FALSE)

}

```

```

# Combine all metrics into a single table
metrics_table <- data.frame(
  Accuracy = c(acc_RL, acc_LL, acc_RF),
  AUC = c(auc_RL, auc_LL, auc_RF),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
    "Random Forest (RF)")
)
metrics_table |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1	Model
0.5972	0.6583	0.5543	Ridge Logistic (RL)
0.5833	0.6359	0.5533	Lasso Logistic (LL)
0.6111	0.6592	0.5909	Random Forest (RF)

Bootstrap tests for model comparison

```
# Run bootstrapping of accuracy with predictions
B <- 10000
boot_RL <- boot_acc(test$y, y_test_hat_RL, n_boot = B, seed = 123)
boot_LL <- boot_acc(test$y, y_test_hat_LL, n_boot = B, seed = 123)
boot_RF <- boot_acc(test$y, y_test_hat_RF, n_boot = B, seed = 123)
boot <- list(RL = boot_RL, LL = boot_LL, RF = boot_RF)

# Bootstrap accuracy differences between RF and other models
diff_boot_RF_RL <- boot$RF - boot$RL
diff_boot_RF_LL <- boot$RF - boot$LL

# One-sided p-values (H1: RF better than other model)
p_RF_vs_RL <- (1 + sum(diff_boot_RF_RL <= 0)) / (B + 1)
p_RF_vs_LL <- (1 + sum(diff_boot_RF_LL <= 0)) / (B + 1)

# Two-sided 95% confidence intervals for differences
ci_diff_RL <- quantile(diff_boot_RF_RL, probs = c(0.025, 0.975))
ci_diff_LL <- quantile(diff_boot_RF_LL, probs = c(0.025, 0.975))

# Add bootstrap tests to metrics table
metrics_table <- metrics_table |>
  mutate(
    ci_diff_lo = c(ci_diff_RL[1], ci_diff_LL[1], NA),
    ci_diff_up = c(ci_diff_RL[2], ci_diff_LL[2], NA),
    pval_better = c(p_RF_vs_RL, p_RF_vs_LL, NA)
  )
metrics_table |>
  kable(digits = 4)
```

Accuracy	AUC	Macro_F1	Model	ci_diff_lo	ci_diff_up	pval_better
0.5972	0.6583	0.5543	Ridge Logistic (RL)	-0.0463	0.0741	0.3543
0.5833	0.6359	0.5533	Lasso Logistic (LL)	-0.0417	0.0972	0.2364
0.6111	0.6592	0.5909	Random Forest (RF)	NA	NA	NA

Comparison with majority class baseline

```
# Majority Class (MC) prediction baseline - always predict most common class
class_probs <- table(train$y) / length(train$y)
majority_class_train <- names(sort(class_probs, decreasing = TRUE))[1]
y_test_hat_MC <- factor(rep(majority_class_train, length(test$y)),
  levels = levels(test$y))
```

```

# Metrics for majority class baseline
acc_MC <- mean(test$y == y_test_hat_MC)
p_test_hat_MC <- rep(class_probs[majority_class_train], length(test$y))
auc_MC <- auc_binary(test$y, p_test_hat_MC)
f1_MC <- f1(test$y, y_test_hat_MC)

# Bootstrap accuracy for majority class baseline
boot_MC <- boot_acc(test$y, y_test_hat_MC, n_boot = B, seed = 123)

# Bootstrap accuracy differences between majority class baseline and models
diff_boot_RL_MC <- boot_RL - boot_MC
diff_boot_LL_MC <- boot_LL - boot_MC
diff_boot_RF_MC <- boot_RF - boot_MC

# One-sided p-values (H1: model better than majority class baseline)
p_RL_vs_MC <- (1 + sum(diff_boot_RL_MC <= 0)) / (B + 1)
p_LL_vs_MC <- (1 + sum(diff_boot_LL_MC <= 0)) / (B + 1)
p_RF_vs_MC <- (1 + sum(diff_boot_RF_MC <= 0)) / (B + 1)

# Two-sided 95% confidence intervals for differences
ci_diff_RL_MC <- quantile(diff_boot_RL_MC, probs = c(0.025, 0.975))
ci_diff_LL_MC <- quantile(diff_boot_LL_MC, probs = c(0.025, 0.975))
ci_diff_RF_MC <- quantile(diff_boot_RF_MC, probs = c(0.025, 0.975))

# Combine all metrics into a single table
metrics_table_MC <- data.frame(
  Accuracy = c(acc_RL, acc_LL, acc_RF, acc_MC),
  AUC = c(auc_RL, auc_LL, auc_RF, auc_MC),
  Macro_F1 = c(f1_RL, f1_LL, f1_RF, f1_MC),
  ci_diff_lo = c(ci_diff_RL_MC[1], ci_diff_LL_MC[1], ci_diff_RF_MC[1], NA),
  ci_diff_up = c(ci_diff_RL_MC[2], ci_diff_LL_MC[2], ci_diff_RF_MC[2], NA),
  pval_better = c(p_RL_vs_MC, p_LL_vs_MC, p_RF_vs_MC, NA),
  Model = c("Ridge Logistic (RL)", "Lasso Logistic (LL)",
            "Random Forest (RF)", "Majority Class (MC)")
)
metrics_table_MC |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1	ci_diff_lo	ci_diff_up	pval_better	Model
0.5972	0.6583	0.5543	-0.0231	0.1065	0.1209	Ridge Logistic (RL)
0.5833	0.6359	0.5533	-0.0417	0.1019	0.2461	Lasso Logistic (LL)
0.6111	0.6592	0.5909	-0.0185	0.1343	0.0843	Random Forest (RF)
0.5556	0.5000	0.3571	NA	NA	NA	Majority Class (MC)

```
# In a combined table
metrics_table_combined <- metrics_table_MC |>
  mutate(ci_diff_lo_MC = ci_diff_lo, ci_diff_up_MC = ci_diff_up,
         pval_better_MC = pval_better) |>
  mutate(ci_diff_lo_RF = c(metrics_table$ci_diff_lo, NA),
         ci_diff_up_RF = c(metrics_table$ci_diff_up, NA),
         pval_better_RF = c(metrics_table$pval_better, NA)) |>
  select(Model, Accuracy, AUC, Macro_F1,
         ci_diff_lo_RF, ci_diff_up_RF, pval_better_RF,
         ci_diff_lo_MC, ci_diff_up_MC, pval_better_MC)
metrics_table_combined |>
  kable(digits = 4)
```

Model	Accu- racy	AUC	Macro_F1	ci_diff_lo_RF	ci_diff_up_RF	pval_bet- ter_RF	ci_diff_lo_MC	ci_diff_up_MC	pval_bet- ter_MC
Ridge	0.5972	0.6583	0.5543	-0.0463	0.0741	0.3543	-0.0231	0.1065	0.1209
Logistic (RL)									
Lasso	0.5833	0.6359	0.5533	-0.0417	0.0972	0.2364	-0.0417	0.1019	0.2461
Logistic (LL)									
Random Forest (RF)	0.6111	0.6592	0.5909	NA	NA	NA	-0.0185	0.1343	0.0843
Majority Class (MC)	0.5556	0.5000	0.3571	NA	NA	NA	NA	NA	NA

```
# Save table of all metrics + bootstrap tests
dir.create("results/experiments", showWarnings = FALSE)
write.csv(metrics_table_combined, paste0("results/experiments/all_results_",
                                         "Experiment_", experiment,
                                         "_sub_", subset_name, ".csv"),
          row.names = FALSE)
```

```
# Print LaTeX table
latex_table <- metrics_table_combined |>
  mutate(ci_RF = paste0("$(", sprintf("%.4f", ci_diff_lo_RF), ", ",
                                   sprintf("%.4f", ci_diff_up_RF), ")$"),
         ci_MC = paste0("$(", sprintf("%.4f", ci_diff_lo_MC), ", ",
                                   sprintf("%.4f", ci_diff_up_MC), ")$"),
         p_RF = sprintf("%.4f", pval_better_RF),
         p_MC = sprintf("%.4f", pval_better_MC)) |>
  select(Model, Accuracy, AUC, Macro_F1, ci_RF, p_RF, ci_MC, p_MC)
latex_table |>
  kable(digits = 4)
```

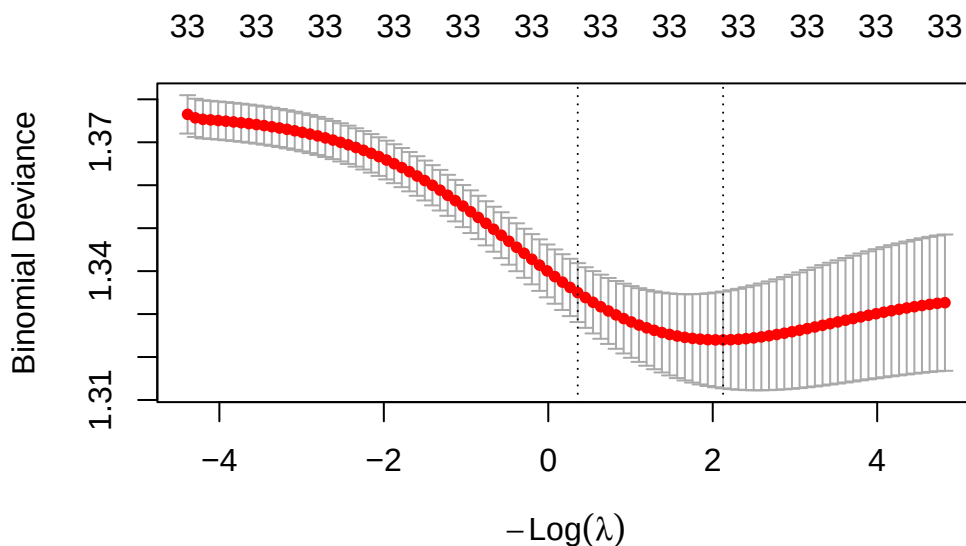
Model	Accuracy	AUC	Macro_F1	ci_RF	p_RF	ci_MC	p_MC
Ridge Logistic (RL)	0.5972	0.6583	0.5543	(-0.0463, 0.0741)	0.3543	(-0.0231, 0.1065)	0.1209
Lasso Logistic (LL)	0.5833	0.6359	0.5533	(-0.0417, 0.0972)	0.2364	(-0.0417, 0.1019)	0.2461
Random Forest (RF)	0.6111	0.6592	0.5909	(NA, NA)	NA	(-0.0185, 0.1343)	0.0843
Majority Class (MC)	0.5556	0.5000	0.3571	(NA, NA)	NA	(NA, NA)	NA

```
latex_table |>
  kable(digits = 4, format = "latex", booktabs = TRUE, escape = FALSE) |>
  print()
```

```
\begin{tabular}{lrrrrllll}
\toprule
Model & Accuracy & AUC & Macro_F1 & ci_RF & p_RF & ci_MC & p_MC\\
\midrule
Ridge Logistic (RL) & 0.5972 & 0.6583 & 0.5543 &  $\$(-0.0463, 0.0741)\$$  & 0.3543 &  $\$(-0.0231, 0.1065)\$$  & 0.1209\\
Lasso Logistic (LL) & 0.5833 & 0.6359 & 0.5533 &  $\$(-0.0417, 0.0972)\$$  & 0.2364 &  $\$(-0.0417, 0.1019)\$$  & 0.2461\\
Random Forest (RF) & 0.6111 & 0.6592 & 0.5909 &  $\$(NA, NA)\$$  & NA &  $\$(-0.0185, 0.1343)\$$  & 0.0843\\
Majority Class (MC) & 0.5556 & 0.5000 & 0.3571 &  $\$(NA, NA)\$$  & NA &  $\$(NA, NA)\$$  & NA\\
\bottomrule
\end{tabular}
```

Ridge logistic for inference and final model on all data (train + test)

```
# Fit ridge
set.seed(42)
all_bake_no_y <- all_bake |>
  select(-y) |>
  as.matrix()
cv0_all <- cv.glmnet(x = all_bake_no_y, y = all_bake$y, family = "binomial",
  alpha = 0, nfolds = 10)
plot(cv0_all)
```



```
cv0_all
```

Call: `cv.glmnet(x = all_bake_no_y, y = all_bake$y, nfolds = 10, family = "binomial", alpha = 0)`

Measure: Binomial Deviance

	Lambda	Index	Measure	SE	Nonzero
min	0.1193	71	1.324	0.01127	33
1se	0.6987	52	1.335	0.00686	33

```
# Confidence intervals for all coefficients
ci_df <- logistic_ridge_ci(X = all_bake_no_y, y = all_bake$y,
                           beta_hat = coef(cv0_all, s = cv0_all$lambda.1se),
                           lambda = nrow(all_bake) * cv0_all$lambda.1se / 2)

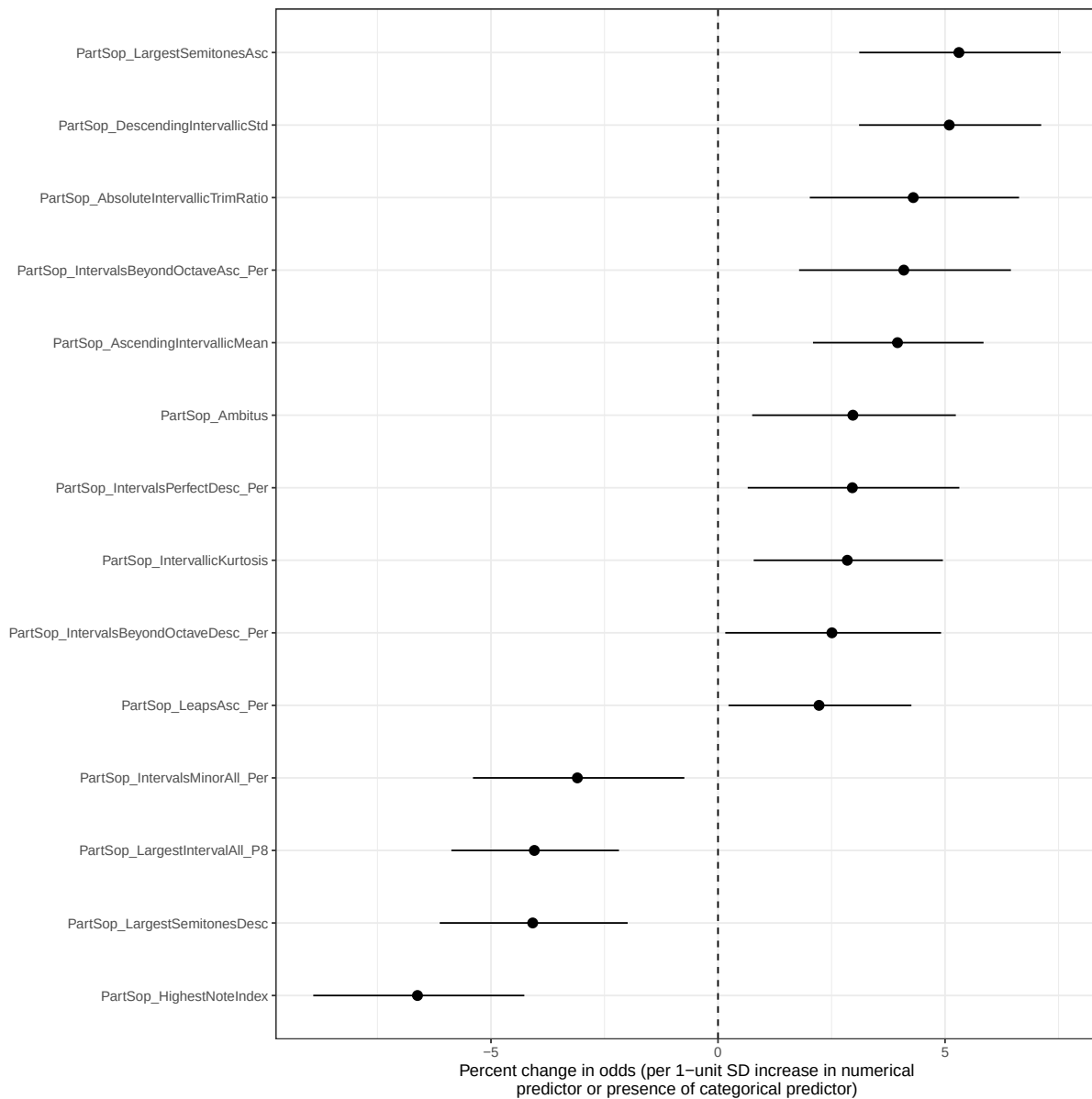
# Filter significant coefficients and order by effect size
ci_df <- ci_df |>
  filter(sign(ci_low * ci_up) > 0) |>
  mutate(
    pct_change = (exp(estimate) - 1) * 100,
    pct_low = (exp(ci_low) - 1) * 100,
    pct_up = (exp(ci_up) - 1) * 100
  ) |>
  mutate(
    term = factor(term, levels = term[order(pct_change, decreasing = FALSE)])
  )
ci_df |>
  select(term, pct_change, pct_low, pct_up, p_value) |>
  arrange(desc(pct_change)) |>
  kable(digits = 2)
```

term	pct_change	pct_low	pct_up	p_value
PartSop_LargestSemitonesAsc	5.30	3.11	7.55	0.00
PartSop_DescendingIntervallicStd	5.09	3.11	7.12	0.00
PartSop_AbsoluteIntervallicTrimRatio	4.30	2.02	6.63	0.00
PartSop_IntervalsBeyondOctaveAsc_Per	4.09	1.79	6.45	0.00
PartSop_AscendingIntervallicMean	3.95	2.09	5.85	0.00
PartSop_Ambitus	2.97	0.75	5.24	0.01
PartSop_IntervalsPerfectDesc_Per	2.96	0.65	5.31	0.01
PartSop_IntervallicKurtosis	2.85	0.79	4.95	0.01
PartSop_IntervalsBeyondOctaveDesc_Per	2.51	0.16	4.91	0.04
PartSop_LeapsAsc_Per	2.22	0.23	4.26	0.03
PartSop_IntervalsMinorAll_Per	-3.10	-5.40	-0.74	0.01
PartSop_LargestIntervalAll_P8	-4.04	-5.87	-2.18	0.00
PartSop_LargestSemitonesDesc	-4.08	-6.13	-1.99	0.00
PartSop_HighestNoteIndex	-6.62	-8.91	-4.27	0.00

```

# Plot of coefficients in odds
plot_RL <- ggplot(ci_df, aes(x = term, y = pct_change,
                             ymin = pct_low, ymax = pct_up)) +
  geom_pointrange() +
  geom_hline(yintercept = 0, linetype = 2) +
  coord_flip() +
  labs(
    x = NULL,
    y = "Percent change in odds (per 1-unit SD increase in numerical\npredictor or presence of category)"
  ) +
  theme_bw()
plot_RL

```



```
# Save plot
plot_name <- paste0("plots/plot_rl", "_", experiment,
                    "_sub_", subset_name, ".png")
ggsave(filename = plot_name, plot = plot_RL, width = 8, height = 6, dpi = 300)

# Metrics on all data available, just for reference
y_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
                        type = "class")
p_all_hat_RL <- predict(cv0_all, newx = all_bake_no_y, s = "lambda.1se",
                        type = "response")
```

```

acc_all_RL <- mean(all_bake$y == y_all_hat_RL)
auc_all_RL <- auc_binary(all_bake$y, p_all_hat_RL)
f1_all_RL <- f1(all_bake$y, y_all_hat_RL)
glmnet_summary <- data.frame("Accuracy" = acc_all_RL, "AUC" = auc_all_RL,
                             "Macro_F1" = f1_all_RL,
                             "Model" = "Ridge Logistic (RL)")

glmnet_summary |>
  kable(digits = 4)

```

Accuracy	AUC	Macro_F1	Model
0.6058	0.6466	0.5529	Ridge Logistic (RL)

```

# Save results
dir.create("results/final_model", showWarnings = FALSE)
write.csv(glmnet_summary, paste0("results/final_model/RL_final_model_results",
                                "_Experiment_", experiment, "_sub_",
                                subset_name, ".csv"),
          row.names = FALSE)

```